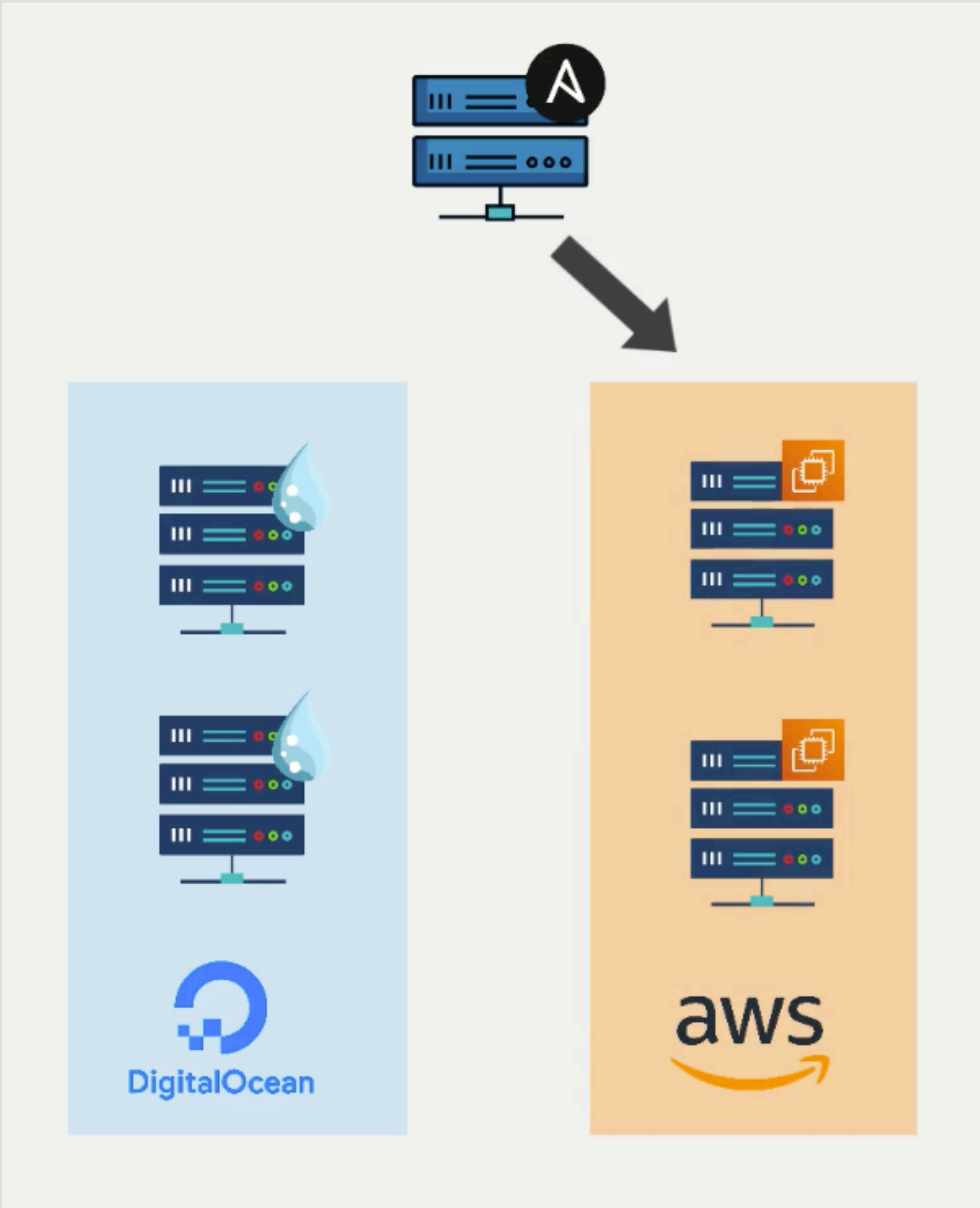
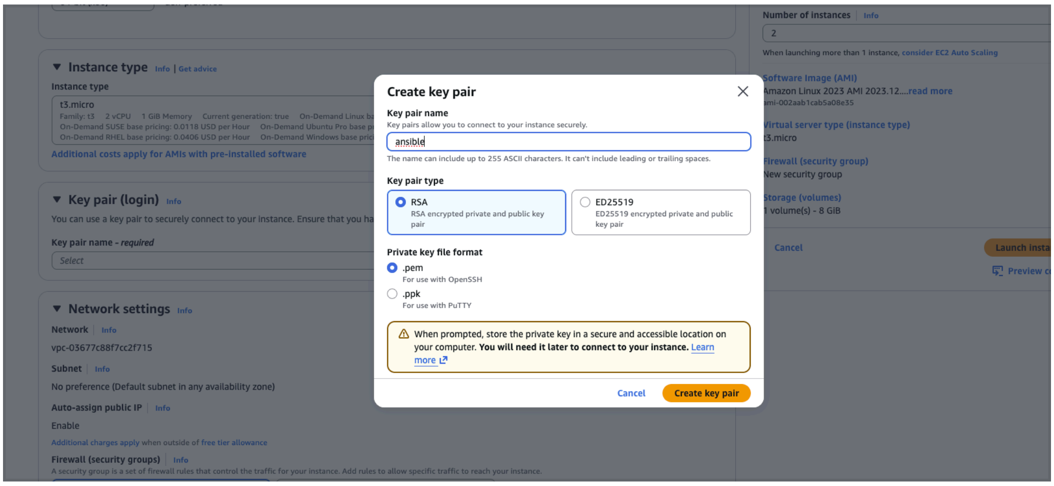
Module 15 - chapter 5

Configure AWS EC2 server with Ansible

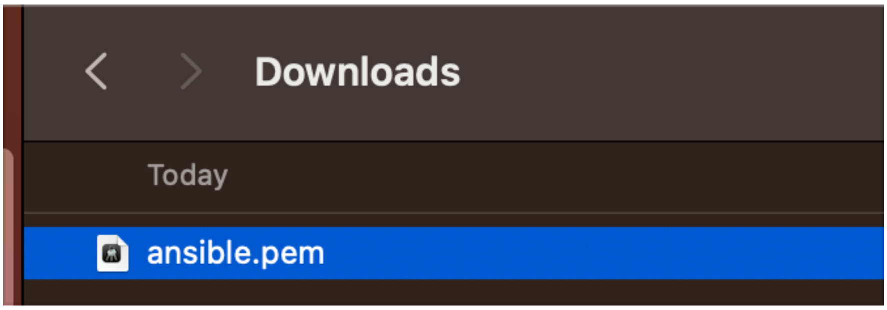
Now lets' add two EC2 servers



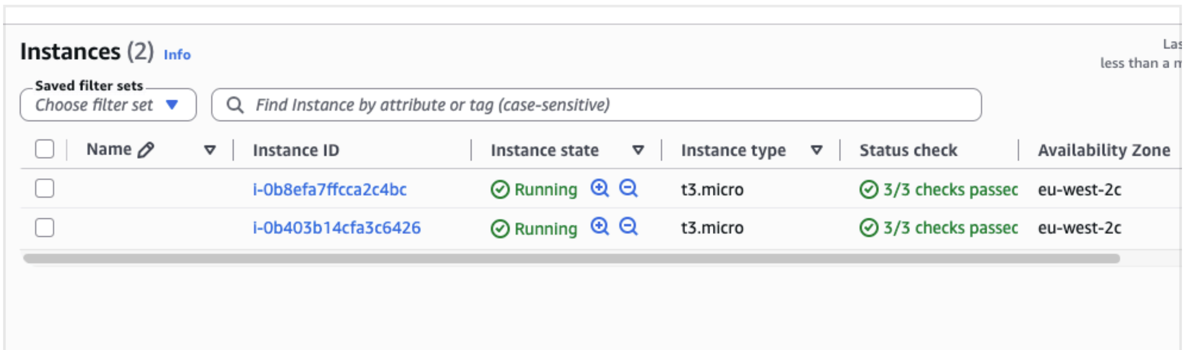
So we will launch T3.micro EC2 instances and we will create a new key pair with AWS to ssh into the instances. So we download the private key.



And stored here:
-> ~/Downloads/ansible.pem

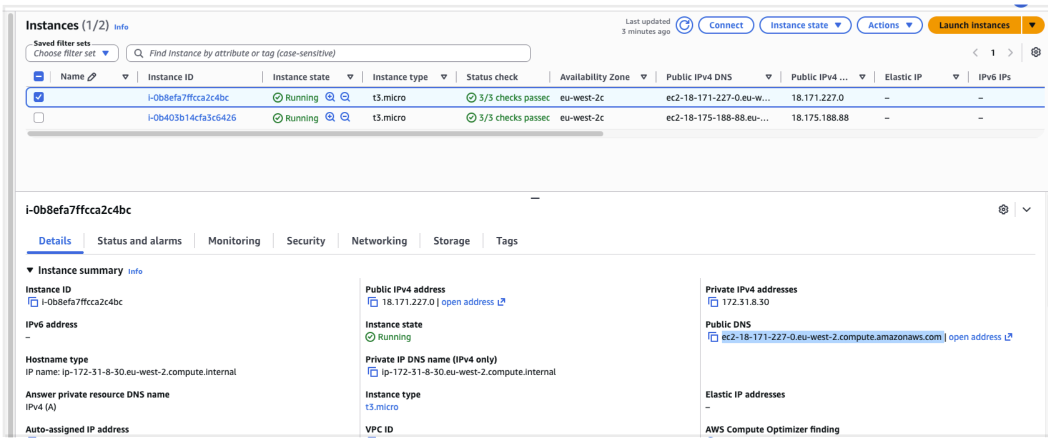


And we launch the instance s



Now we can configure the two AWS servers in the host file:

- And in AWs in addition to the IP address we also get the hostnames, the DNS names. So we will use them



And we update 'hosts' file

```
vim (vim)                                ⌘1      ✳ Understand Ansible alternatives to Docke
[droplet]
46.101.88.242
46.101.80.55

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_id_ed255192
ansible_user=root

[ec2]
ec2-18-171-227-0.eu-west-2.compute.amazonaws.com
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com

[ec2:vars]
ansible_ssh_private_key_file=~/Downloads/ansible.pem
ansible_user=ec2-user
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- INSERT --
```

```
~ (-zsh)                                   ⌘1    ✳ Understand Ansible alte
❯ vim hosts
❯ cat hosts
[droplet]
46.101.88.242
46.101.80.55

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_id_ed255192
ansible_user=root

[ec2]
ec2-18-171-227-0.eu-west-2.compute.amazonaws.com
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com

[ec2:vars]
ansible_ssh_private_key_file=~/Downloads/ansible.pem
ansible_user=ec2-user
❯
 ~
```

Default User for EC2 instances Is ‘ec2-user’

And we need to edit the permissions for the file ansible.pem where only my user has access and only read access -> BEST practice

AWS .pem files usually need restrictive permissions or SSH will refuse to use them for auth:
chmod 400 ~/Downloads/ansible.pem

So
-> ls -l ~/Downloads/ansible.pem

```
❯ ls -l ~/Downloads/ansible.pem
-rw-r--r--@ 1 astrid  staff  1678 14 Jul 13:59 /Users/astrid/Downloads/ansible.pem
❯
```

-> chmod 400 ~/Downloads/ansible.pem
-> ls -l ~/Downloads/ansible.pem

```
❯ chmod 400 ~/Downloads/ansible.pem
❯ ls -l ~/Downloads/ansible.pem
-r--------@ 1 astrid  staff  1678 14 Jul 13:59 /Users/astrid/Downloads/ansible.pem
❯
 ~ ❯
```

Now let’s test connection to one of the instances:
-> ssh -i ~/Downloads/ansible.pem ec2-user@ec2-18-175-188-88.eu-west-2.compute.amazonaws.com

```
❯ ssh -i ~/Downloads/ansible.pem ec2-user@ec2-18-175-188-88.eu-west-2.compute.amazonaws.com
The authenticity of host 'ec2-18-175-188-88.eu-west-2.compute.amazonaws.com (18.175.188.88)' can't be established.
ED25519 key fingerprint is SHA256:6xBKm2m70ld51Wu1p5dXH/qp7SXAUADlLygg/GKZDsA.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-18-175-188-88.eu-west-2.compute.amazonaws.com' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-11-184 ~]$
```

And we can see the user is ‘ec2-user’

And let’s see if the instance has python installed
-> python3

```
       _/m/'
[ec2-user@ip-172-31-11-184 ~]$ python3
Python 3.9.25 (main, Jun 17 2026, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Now let’s try to execute the ad-hoc ansible command:
-> ansible ec2 -i hosts -m ping

```
❯ ansible ec2 -i hosts -m ping
[ERROR]: Task failed: Failed to connect to the host via ssh: Host key verification failed.
Origin: <adhoc 'ping' task>

{'action': 'ping', 'args': {}, 'timeout': 0, 'async_val': 0, 'poll': 15}

ec2-18-171-227-0.eu-west-2.compute.amazonaws.com | UNREACHABLE! => {
    "changed": false,
    "msg": "Task failed: Failed to connect to the host via ssh: Host key verification failed.",
    "unreachable": true
}
[WARNING]: Host 'ec2-18-175-188-88.eu-west-2.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/python
3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs
.ansible.com/ansible-core/2.21/reference_appendices/interpreter_discovery.html for more information.
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
```

Same problem as with DigitalOcean
So I need to
-> ssh-keyscan ec2-18-171-227-0.eu-west-2.compute.amazonaws.com >> ~/.ssh/known_hosts

```
❯ ansible ec2 -i hosts -m ping
[ERROR]: Task failed: Failed to connect to the host via ssh: Host key verification failed.
Origin: <adhoc 'ping' task>

{'action': 'ping', 'args': {}, 'timeout': 0, 'async_val': 0, 'poll': 15}

ec2-18-171-227-0.eu-west-2.compute.amazonaws.com | UNREACHABLE! => {
    "changed": false,
    "msg": "Task failed: Failed to connect to the host via ssh: Host key verification faile
    "unreachable": true
}
[WARNING]: Host 'ec2-18-175-188-88.eu-west-2.compute.amazonaws.com' is using the discovered
3.9', but future installation of another Python interpreter could cause a different interpr
.ansible.com/ansible-core/2.21/reference_appendices/interpreter_discovery.html for more inf
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
❯ ssh-keyscan ec2-18-171-227-0.eu-west-2.compute.amazonaws.com >> ~/.ssh/known_hosts
# ec2-18-171-227-0.eu-west-2.compute.amazonaws.com:22 SSH-2.0-OpenSSH_8.7
# ec2-18-171-227-0.eu-west-2.compute.amazonaws.com:22 SSH-2.0-OpenSSH_8.7
# ec2-18-171-227-0.eu-west-2.compute.amazonaws.com:22 SSH-2.0-OpenSSH_8.7
# ec2-18-171-227-0.eu-west-2.compute.amazonaws.com:22 SSH-2.0-OpenSSH_8.7
# ec2-18-171-227-0.eu-west-2.compute.amazonaws.com:22 SSH-2.0-OpenSSH_8.7
❯
```

And try again
-> ansible ec2 -i hosts -m ping

```
❯ ansible ec2 -i hosts -m ping
[WARNING]: Host 'ec2-18-175-188-88.eu-west-2.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/python
3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs
.ansible.com/ansible-core/2.21/reference_appendices/interpreter_discovery.html for more information.
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Host 'ec2-18-171-227-0.eu-west-2.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/python3
.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.
ansible.com/ansible-core/2.21/reference_appendices/interpreter_discovery.html for more information.
ec2-18-171-227-0.eu-west-2.compute.amazonaws.com | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
```

We got access and also we got a warning:
"[WARNING]: Host 'ec2-18-171-227-0.eu-west-2.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference_appendices/interpreter_discovery.html for more information."

And this is the explanation of the warning:

Why Ansible needs to "discover" Python at all

Most Ansible modules (everything except raw/script/a few others) aren't shell commands — they're Python scripts that Ansible copies to the target and executes there. So before it can run anything, Ansible needs to know: which Python binary should I use on this specific machine?

You never told it (no ansible_python_interpreter set in your inventory), so Ansible ran its interpreter discovery routine — checks a list of common paths/binaries on the target — and found /usr/bin/python3.9. It used that, and it worked (discovered_interpreter_python in the output confirms which one it picked).

Why the warning exists

It's Ansible flagging a risk with auto-discovery specifically: it's not a fixed setting, it's re-evaluated based on what's installed. If someone later installs a different Python version on that EC2 box (e.g. python3.11 via a system update or manual install), a future run might auto-discover a different interpreter than this one — silently changing which Python your modules execute under, potentially with different behavior/available packages. It's a "this works today, but the ground could shift under you" warning, not a current problem.

How to make it go away / pin it down

Set **ansible_python_interpreter** explicitly in your inventory, same way you set ansible_user.

This tells Ansible "**always use this exact interpreter, don't guess**" — removes the ambiguity and the warning, and guarantees consistent behavior across runs regardless of what else gets installed on the box later. Good practice once you move past ad-hoc ping testing into real playbooks.

So let's override the default behaviour of discovering the python interpreter and instead we set the python we want to use in this case the current one the EC2 has python3.9 -> /usr/bin/python3.9

```
[droplet]
157.230.29.95
157.230.29.113

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_user=root

[ec2]
ec2-18-192-209-223.eu-central-1.compute.amazonaws.com
ec2-18-193-88-102.eu-central-1.compute.amazonaws.com ansible_python_interpreter=/usr/bin/python3.9

[ec2:vars]
ansible_ssh_private_key_file=~/Downloads/ansible.pem
ansible_user=ec2-user
```

So this is my hosts now:

```
[droplet]
46.101.88.242
46.101.80.55

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_id_ed255192
ansible_user=root

[ec2]
ec2-18-171-227-0.eu-west-2.compute.amazonaws.com
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com

[ec2:vars]
ansible_ssh_private_key_file=~/Downloads/ansible.pem
ansible_user=ec2-user
ansible_python_interpreter=/usr/bin/python3.9
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- INSERT --
```

```
❯ vim hosts
❯ cat hosts
[droplet]
46.101.88.242
46.101.80.55

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_id_ed255192
ansible_user=root

[ec2]
ec2-18-171-227-0.eu-west-2.compute.amazonaws.com
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com

[ec2:vars]
ansible_ssh_private_key_file=~/Downloads/ansible.pem
ansible_user=ec2-user
ansible_python_interpreter=/usr/bin/python3.9
❯
 ~
```

And execute ansible ad-hoc command again
-> ansible ec2 -i hosts -m ping

```
❯ ansible ec2 -i hosts -m ping
ec2-18-171-227-0.eu-west-2.compute.amazonaws.com | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
ec2-18-175-188-88.eu-west-2.compute.amazonaws.com | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

And no warning.